Cairo University
Faculty of Engineering
Department of Computer Engineering

Graph-Based Machine Learning Pipelines and Automatic Loop Parallelization in Accelera

Accelera Pipe and Parallelizer Modules

A Graduation Project Individual Report Submitted
to
Faculty of Engineering, Cairo University
in Partial Fulfillment of the Requirements for the Degree
of
Bachelor of Science in Computer Engineering.

Presented by

Mohamed Ashraf
ID: 9220658

Supervised by

Dr. Salma Abdel Monem

June 2026

Abstract

This paper describes two core modules of Accelera: accelera_pipe, a graph-based machine learning pipeline executor, and the Parallelizer, a source-to-source loop optimization module that emits OpenMP-enabled C++ for supported code patterns. The pipeline module represents preprocessing, model fitting, prediction, metrics, branching, merging, and executed inference graphs as a native backend graph. This representation enables parallel branch execution, path selection, graph reuse, and targeted memory optimizations. The Parallelizer converts supported Python code to C++, extracts loops, classifies safe OpenMP transformations, inserts pragmas, and can compile custom preprocessing functions into Python-callable native extensions. Experiments show that accelera_pipe preserves sklearn-equivalent accuracy while reducing latency on a large four-candidate model-selection benchmark from about 803 seconds to 495 seconds. The Parallelizer also accelerates hard loop examples and reduces a preprocessing-heavy Python kernel from 6.83 seconds to 0.08 seconds in a repeated cached end-to-end pipeline run.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Modern machine learning workflows often require repeated evaluation of several preprocessing and model candidates. A direct implementation usually fits each candidate pipeline independently, repeats transformations, and retains intermediate arrays longer than necessary. In parallel, user-defined preprocessing functions may contain numeric loops written in Python, which can become a major bottleneck when the input data grows.

The project can be viewed as solving two connected execution problems:

- **Workflow-level inefficiency:** repeated preprocessing, duplicated branch execution, and unclear intermediate-data lifetime.
- **Code-level inefficiency:** slow custom Python preprocessing loops that could benefit from native compilation and OpenMP parallel execution.

Table 1.1 summarizes the main scope of the modules covered by this paper.

Table 1.1: Main module scope in this report.

Module	Main idea	Expected benefit
accelera_pipe	Represent machine-learning workflows as a graph of preprocessing, model, prediction, metric, merge, and branch nodes.	Avoids purely sequential workflow execution and supports reusable executed graphs.
Parallelizer	Analyze supported Python/C++ loops, insert safe OpenMP pragmas, and compile optimized native functions when possible.	Speeds up eligible loop-heavy custom preprocessing code while keeping fallback behavior safe.
Integration path	Connect the Parallelizer to accelera_pipe preprocessing nodes.	Allows pipeline-level execution and code-level acceleration to work together.

Accelera addresses these two issues through two complementary modules. The first module, accelera_pipe, expresses machine learning workflows as a graph. Nodes represent preprocessing steps, models, predictions, metrics, merges, and branch alternatives. The graph backend executes independent branches in parallel and returns an executed graph that can be reused for inference. This allows the system to separate training-time

exploration from inference-time execution. During training, the graph may evaluate several branches and keep temporary validation-related data, while during inference it only needs the selected fitted path.

The second module, the Parallelizer, analyzes loop-heavy source code and adds OpenMP pragmas when the loop structure is considered safe. For supported Python functions, a restricted Python-to-C++ converter emits C++ code before loop extraction. The generated code can then be compiled into a Python extension and inserted into an accelera_pipe preprocessing node. This gives the system a fallback-safe optimization path: unsupported code remains executable in its original Python form, while supported numeric loops can be replaced with a native implementation.

The integration between the two modules is important because workflow optimization and code optimization solve different parts of the same execution problem. A graph executor can reduce duplicated work between branches, but it cannot automatically make a slow custom Python loop faster. Similarly, a code parallelizer can accelerate a loop, but it does not manage pipeline branching, metric selection, or executed-graph reuse. Combining both modules allows Accelera to optimize both the structure of the workflow and the execution of eligible custom operations inside that workflow.

This paper focuses only on these two modules. The main contributions described here are: a graph-based pipeline execution model, memory and branching optimizations for pipeline search, a hybrid rule and classifier loop-parallelization strategy, a documented Python-to-C++ subset, and an integration path that uses the Parallelizer as an acceleration backend for custom preprocessing functions.

Chapter 2

Literature Survey

This chapter reviews the main background and previous work related to the Accelera project. The chapter is divided into four main parts. First, it introduces machine learning pipeline frameworks, with special attention to scikit-learn, which is the main practical baseline for the accelera_pipe module. Second, it explains the background of automatic code parallelization and OpenMP, which are needed to understand the design of the Parallelizer module. Third, it compares the project with related work, especially scikit-learn pipelines and OMPar, an AI-driven OpenMP parallelization system. Finally, it explains the implemented approach adopted in this project and justifies the main design choices, including the use of rule-based pragma generation instead of fully AI-generated pragma insertion.

The chapter organization is summarized in Table 2.1.

Table 2.1: Literature-survey organization.

Part	Purpose
Machine-learning pipeline frameworks	Explains why pipeline abstractions are useful and why scikit-learn is the main baseline for accelera_pipe.
Automatic parallelization and OpenMP	Introduces loop-level parallelization, dependency safety, reductions, and OpenMP pragmas.
Comparative study	Compares Accelera with scikit-learn and OMPar, focusing on what is borrowed and what is changed.
Implemented approach	Explains why the project combines graph execution with conservative rule-based OpenMP generation.

2.1 Background on Machine Learning Pipeline Frameworks

Machine learning projects rarely consist of a single model call. A complete workflow usually includes data loading, preprocessing, feature transformation, model fitting, prediction, metric evaluation, and sometimes several alternative branches that compare different preprocessing or model choices. Without a structured pipeline abstraction, these steps can become hard to reproduce and maintain. For example, preprocessing may accidentally

be applied differently during training and testing, or several experimental branches may repeat the same early computation many times.

A common Python solution for this problem is the pipeline abstraction provided by scikit-learn [1]. In scikit-learn, a pipeline connects a sequence of transformers followed by a final estimator. A linear pipeline can be represented as:

$$P_{\text{linear}} = f_k \circ f_{k-1} \circ \cdots \circ f_1$$

where each function f_i represents one preprocessing or modelling step. The user can then fit, predict, and evaluate the complete chain through one high-level object. This design is important because it reduces user error, keeps preprocessing attached to the model, and makes experiments easier to reproduce. It also provides a familiar interface for combining preprocessing operations such as scaling, encoding, or feature selection with estimators such as logistic regression, support vector machines, decision trees, or other machine learning models.

The role of scikit-learn in this project is therefore central. It is the main competitor and baseline for the accelera_pipe module. The comparison is natural because both systems allow users to describe a machine learning workflow in Python and both support common preprocessing and modelling components. However, the two systems make different execution assumptions. A traditional scikit-learn pipeline is mainly a linear composition of steps. It is very effective when the user wants to express one selected path, such as scaling followed by logistic regression. However, it does not directly represent the workflow as a native execution graph that can share intermediate results across several candidate branches.

This difference becomes more important when the user wants to compare several alternatives. In a traditional workflow, the user may create multiple pipelines or use a model-selection utility that repeatedly fits different candidate configurations. Although this approach is simple and reliable, it can duplicate work. For example, if several candidate models all use the same preprocessing output, the preprocessing stage may be recomputed or stored separately for each candidate path. This repeated work can increase runtime and memory usage, especially when the dataset is large or when the preprocessing step is expensive.

The motivation for accelera_pipe is to keep the familiar pipeline idea while changing the internal representation from a simple chain into a graph. In this graph, nodes represent operations such as preprocessing, modelling, prediction, metric evaluation, merging, and branching. Edges represent data dependencies between these operations. This graph-based representation can be written as:

$$G = (V, E)$$

where V is the set of workflow operations and E is the set of dependency edges between them. This makes the workflow closer to a computation graph than to a purely sequential list of steps. As a result, the system can expose opportunities for branch scheduling, intermediate result reuse, and memory release after data is no longer needed.

Table 2.2: Comparison between linear and graph-based pipeline execution.

Aspect	**scikit-learn** Pipeline	**accelera_pipe**
Internal structure	Linear sequence of steps.	Directed graph of computational nodes.
Branch handling	Usually handled through separate pipelines or model-selection utilities.	Represented directly inside the graph as alternative paths.
Shared computation	Repeated stages may be recomputed across candidate workflows.	Shared intermediate results can be reused by multiple branches.
Execution model	Mainly sequential pipeline execution.	Graph execution with scheduling and memory-lifetime management.

This project does not aim to replace the scientific reliability or wide ecosystem of scikit-learn. Instead, it treats scikit-learn as a strong baseline and builds on the observation that a graph-based execution model can expose optimization opportunities that are difficult to express in a linear pipeline. Therefore, the experiments compare Accelera against equivalent scikit-learn-style workflows, while the methodology explains how the graph executor schedules independent branches, stores executed graphs, and releases intermediate arrays when their consumers have finished.

2.2 Background on Automatic Code Parallelization

The second important background area is automatic code parallelization. Manual parallelization can improve performance, but it is difficult and error-prone. The programmer must inspect loops, reason about data dependencies, identify possible reductions, select the correct OpenMP directive, and avoid race conditions. A wrong parallelization decision may not only reduce performance, but can also produce incorrect results. This makes automatic parallelization both a performance problem and a correctness problem.

The expected performance benefit of parallelization is commonly described using speedup:

$$S = \frac{T_{\text{sequential}}}{T_{\text{parallel}}}$$

where $T_{\text{sequential}}$ is the runtime before parallelization and T_{parallel} is the runtime after parallelization. A speedup greater than one means that the parallel version is faster. However, runtime improvement is useful only if the transformed program still produces correct results.

OpenMP is a common target for automatic parallelization because it allows parallel execution to be expressed using compiler directives. In C and C++, directives such as #pragma omp parallel for can be used to distribute loop iterations across multiple threads. This is especially useful in numeric workloads where expensive computation is concentrated inside loops over arrays, matrices, or feature vectors. If loop iterations

are independent, or if the loop matches a safe reduction pattern, the loop can often be parallelized using an OpenMP directive.

However, deciding whether a loop is safe to parallelize is not trivial. A loop may read and write the same variable across iterations, access arrays using indirect indexes, contain branches that exit early, or call functions with hidden side effects. These patterns can create dependencies between iterations, which means that running iterations in parallel may change the result. Therefore, a parallelizer needs more than simple text matching. It must analyze the loop structure, memory accesses, control flow, dependency patterns, and reduction behavior before inserting a pragma.

Table 2.3: Common loop patterns considered during automatic parallelization.

Pattern	**Meaning**	**Parallelization decision**
Independent iterations	Each iteration reads and writes separate data.	Usually safe for parallel for.
Reduction	Iterations update a shared variable using an associative operation such as sum or maximum.	Safe only with a valid OpenMP reduction clause.
Loop-carried dependency	One iteration depends on a value written by another iteration.	Usually unsafe to parallelize directly.
Indirect memory access	Array indexes depend on runtime data.	Requires conservative analysis because dependencies may be hidden.
Side-effect function calls	The loop calls functions that may modify external state.	Usually unsafe unless the function behavior is known.

The Parallelizer module in Accelera is related to this area of work. It analyzes source code, extracts loops, computes loop-level features, and decides whether a safe OpenMP pragma should be inserted. The module also supports a restricted Python-to-C++ conversion path. This allows supported Python functions to be translated into C++ so that they can enter the same loop-analysis and native-compilation pipeline. After analysis, the module can compile the optimized code into a native callable and reuse cached results when the same transformation is needed again. This makes the optimization practical for repeated pipeline runs, not only for one-time source-code rewriting.

This is important for accelera_pipe because the bottleneck in a machine learning workflow is not always the model itself. Sometimes the slow part is a custom preprocessing function written in Python. In that case, graph-level scheduling alone is not enough, the loop-heavy custom code may also need native compilation and parallel execution.

2.3 Comparative Study of Previous Work

The closest practical competitor for accelera_pipe is scikit-learn Pipeline [1]. scikit-learn provides a mature and widely used machine-learning ecosystem in Python. Its pipeline abstraction is simple, well tested, and strongly integrated with the rest of the library, making it a strong baseline for this project.

The main difference is that scikit-learn Pipeline is primarily linear, while accelera_pipe uses a graph-based execution backend. A linear pipeline is suitable for one selected sequence of operations, but branch-heavy workflows may repeat shared preprocessing or intermediate computations. accelera_pipe addresses this by representing the workflow as a graph, storing the executed graph, and using graph-level information to manage execution order and intermediate memory. Therefore, the contribution is not a replacement for scikit-learn, but an execution model for workflows with shared and alternative paths.

For the Parallelizer module, the closest conceptual inspiration is OMPar, short for *Automatic Parallelization with AI-Driven Source-to-Source Compilation* [2]. OMPar proposes an AI-assisted parallelization system for C and C++ programs and separates the task into loop discovery, loop analysis, parallelization decision, and OpenMP pragma generation. This staged design influenced the Parallelizer module in Accelera, which follows a similar flow of loop extraction, feature analysis, classification, and safe code rewriting.

However, Accelera differs from OMPar in pragma generation. OMPar uses an AI-driven pragma generation component, while this project constructs pragma text deterministically and can select recognized patterns before classification. This was chosen because the available labeled data was not sufficient to reliably train a complete pragma-generation model, and synthetic examples may introduce bias.

Unsafe generated pragmas can change program results, so Accelera keeps the final transformation conservative. Safe independent loops receive a parallel-for pragma, recognized reductions receive a reduction pragma, and unclear or unsafe loops are left unchanged. This improves inspectability and reduces the risk of incorrect parallelization.

Table 2.4: Design difference between OMPar and the Parallelizer module.

Aspect	**OMPar**	**Parallelizer in Accelera**
Decision support	Uses AI-assisted analysis and pragma generation.	Uses deterministic rules first and classification only for unresolved loops.
Pragma construction	Generates OpenMP annotations using an AI-driven component.	Builds pragma syntax deterministically for supported heuristic patterns.
Safety strategy	Focuses on AI-driven source-to-source parallelization.	Keeps unsupported or unclear loops unchanged to preserve correctness.

2.4 Implemented Approach

The implemented approach in Accelera combines two levels of optimization: workflow-level optimization and code-level optimization. At the workflow level, accelera_pipe uses a graph representation instead of a purely linear pipeline. This allows preprocessing, model, prediction, metric, merge, and branch operations to be represented as connected nodes. The graph executor can then schedule independent branches, reuse intermediate results, and release data when it is no longer required. This design was chosen because it preserves the high-level usability of pipeline frameworks while providing more control over execution.

At the code level, the Parallelizer module targets loop-heavy preprocessing functions. The system accepts supported input forms, converts supported Python functions into C++, extracts loops using a compiler-style analysis path, computes loop features, applies deterministic rules, classifies unresolved candidates, inserts OpenMP pragmas for selected loops, and compiles the transformed code into a Python-callable native extension. This allows selected custom preprocessing functions to be accelerated while still being called from a Python machine learning workflow.

The connection between accelera_pipe and the Parallelizer is the main implemented contribution. A pipeline node can contain a custom preprocessing function. If that function is supported by the Parallelizer, the system attempts to optimize it by converting and compiling the function. The optimized callable can then be attached back to the pipeline node. In this way, the project does not only optimize the order of pipeline execution, it can also optimize the implementation of selected operations inside the graph.

The project adopts a conservative rule-based pragma generator rather than a fully generative AI pragma generator. This choice was made because the available parallelization data was limited and partly synthetic. A fully generative model could produce unsafe pragmas if it learns biased patterns from synthetic examples. By contrast, a rule-based generator makes the final rewrite easier to control and verify. The implemented resolver first lets deterministic rules select recognized loops, then uses the classifier as a secondary decision source for unresolved loops. Pragma text and supported reduction clauses are always constructed by explicit code rather than generated by a language model.

Accelera combines ideas from scikit-learn pipeline frameworks and OMPar-style automatic parallelization. It uses graph-based workflow execution for machine learning pipelines and conservative rule-based OpenMP acceleration for supported loop-heavy preprocessing code.

Chapter 3

Methodology

3.1 Accelera Pipe Module

3.1.1 Functional Description

The Accelera Pipe module is the high-level workflow construction layer of the system. Its main function is to let the user describe a machine-learning pipeline in Python while delegating graph construction, scheduling, branch execution, and runtime data management to the native C++ backend. Instead of forcing the user to manually manage intermediate outputs between preprocessing, modelling, prediction, and evaluation stages, the module exposes a fluent builder interface where every call adds a computational node to an internal directed acyclic graph (DAG). Formally, the internal pipeline can be represented as:

$$G = (V, E)$$

where V is the set of computational nodes and E is the set of dependency edges between them.

From the user's point of view, the module behaves like a pipeline object. A user can add preprocessing steps, models, prediction nodes, metrics, merge operations, and alternative branches. Internally, however, these operations are not stored as a simple linear list. Each operation is converted into a graph node with a specific type such as `PREPROCESS`, `MODEL`, `PREDICT`, `MERGE`, or `METRIC`. A valid execution order must respect the dependency direction between nodes:

$$u \to v \Rightarrow \tau(u) < \tau(v)$$

where τ is the topological execution order of the graph. This graph-based representation allows the same user-facing API to support sequential workflows, branch evaluation, shared intermediate results, and parallel execution of independent parts of the workflow. It also makes the backend responsible for deciding which nodes are ready to execute, which outputs must be kept, and which temporary results can be released after their final consumer has finished. This separation keeps the Python interface simple while moving execution control to a lower-level implementation that can reason about dependencies more efficiently. As a result, users can write pipeline code in a familiar style, while the system internally maintains a richer representation that supports optimization, persistence, and reuse of the selected executed path.

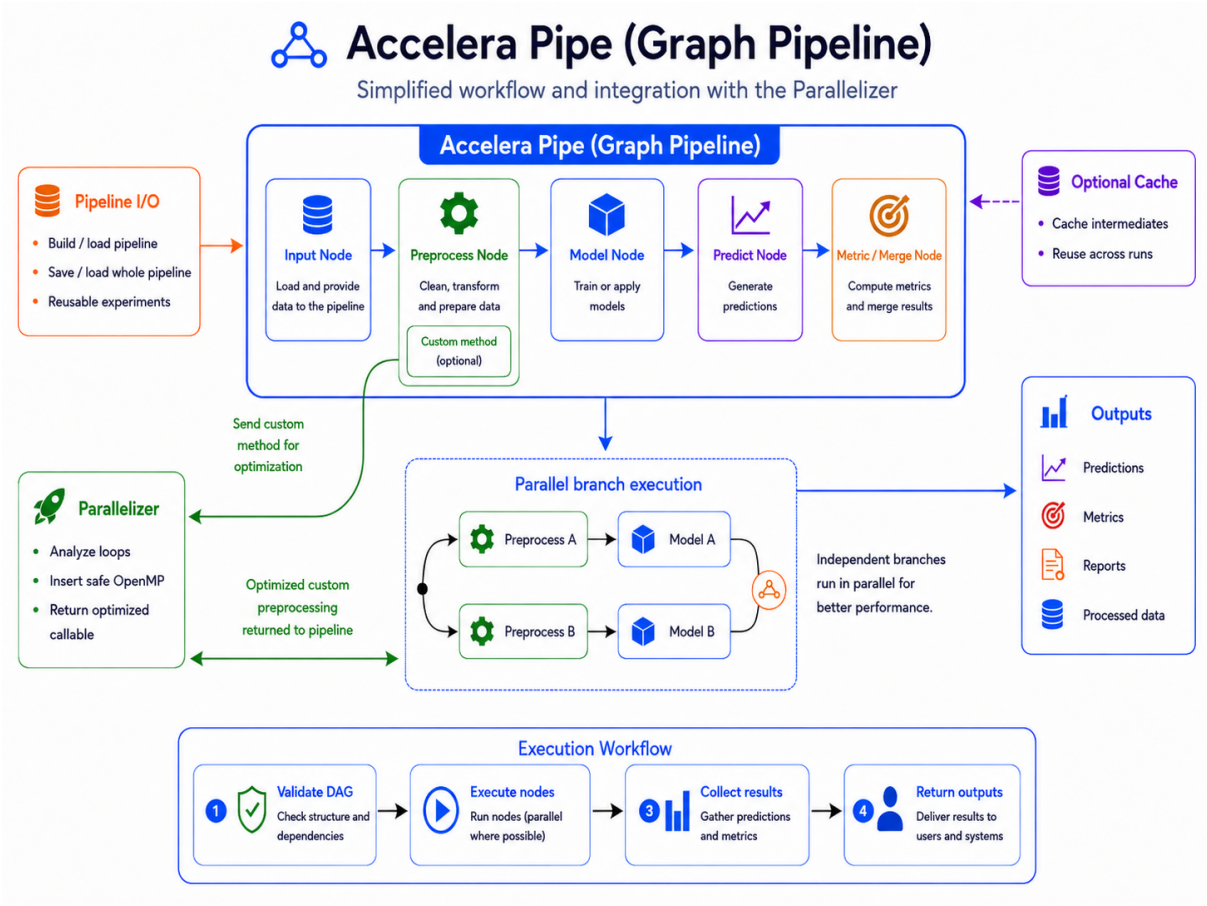


Figure 3.1: Accelera Pipe workflow with custom preprocessing parallelization and pipeline reuse.

As shown in Figure 3.1, the Python layer provides the builder interface and communicates with the native C++ graph engine. The Python layer handles usability, metric wrapping, reports, serialization, save/load support, and integration with the Code Parallelizer. The C++ layer stores nodes and edges, validates graph connections, computes the topological execution order, schedules independent branches, selects the final fitted path, and manages intermediate memory.

The module also separates training-time execution from inference-time execution. When a pipeline is called, the native graph executes all required nodes and returns both the computed predictions or metric results and an `ExecutedGraph` object. The `ExecutedGraph` contains only the selected fitted execution path and can be reused later for inference. For a new input X_{new}, inference can be represented as:

$$\hat{y} = G^*(X_{\text{new}})$$

where G^* is the fitted executed graph selected during training. This separation is practical because training may require validation data, metrics, temporary arrays, and candidate branches, while inference should keep only the fitted transformations and models needed for new data.

Another functional role of Accelera Pipe is transparent optimization of custom preprocessing logic. When the user provides a custom preprocessing function or a custom transformer, the pipeline sends it through the Code Parallelizer before storing it in the

graph. If the transformation is safe and supported, loop-heavy parts can be replaced with a compiled Python-callable C++ implementation. If the optimization cannot be applied, the original function is kept, so the external behavior of the pipeline remains unchanged.

3.1.2 Module Components

Table 3.1: Main components of the Accelera Pipe module

Component	**Responsibility**
Pipeline Builder Interface	Provides the fluent API used to define preprocessing stages, models, prediction steps, metrics, merge operations, and branches. It translates each user call into a typed computational node instead of immediately executing it.
Pipeline State and Persistence Manager	Owns the native graph object, maps high-level node categories to graph node types, controls graph execution settings, and provides save/load behavior for reusable pipeline objects.
Executed Graph Runtime	Represents the fitted execution path selected after training. It allows fitted transformations and models to be reused for inference without keeping validation data or temporary training branches.
Metric and Display Wrappers	Adapt supervised metrics, unsupervised metrics, arrays, figures, dictionaries, and reports into graph-compatible outputs that can be stored, selected, or displayed.
Native Graph Engine	Stores the pipeline as a DAG, validates node connections, computes execution order, schedules sequential or parallel execution, manages branch selection, and controls intermediate-memory lifetime.
Node Abstraction Layer	Defines the common behavior of input, preprocessing, model, prediction, merge, and metric nodes, including source-node relationships, stored data, GPU usage flags, and selected-path markers.

At the Python API level, each builder method creates or configures one graph operation. A preprocessing node stores a function or transformer together with execution parameters such as `execute_fit` and `cache`. Model nodes store estimators, prediction nodes store test data and the prediction method name, metric nodes store metric wrappers, and merge nodes store the merge strategy.

Table 3.2: Mapping from builder calls to graph node semantics

Builder method	Node type	Main role
`preprocess(name, func)`	`PREPROCESS`	Transform input data or selected columns.
`model(name, model)`	`MODEL`	Fit or apply a machine-learning estimator.
`predict(name, test_data)`	`PREDICT`	Run a model output function such as `predict`.
`metric(name, metric)`	`METRIC`	Evaluate predictions using a supervised or unsupervised metric.
`merge(name, strategy)`	`MERGE`	Combine outputs from multiple prediction branches.
`branch(name, ...)`	Multiple nodes	Create alternative candidate paths in the DAG.

3.1.3 Python-to-Native Graph Construction

The implementation divides graph construction between the Python builder in `accelera/src/accelera_pipe/core/pipeline.py` and the C++ graph in `src/core/graph.cpp`. The Python methods do not execute machine learning operations immediately. Instead, each method packages the operation and its options into a Python object and passes that object, a node type, and a name to the PyBind11 graph binding. Table 3.3 shows the concrete payload and runtime output used by each node.

Table 3.3: Concrete payload and output contracts of Accelera Pipe nodes

Node	Stored Python payload	Runtime output
Input	Training or inference values supplied to `Graph.execute`.	Provides `X` and optional `y` to downstream nodes.
Preprocess	`func`, `execute_fit`, and optional `cache`, a column selection is first wrapped in a Scikit-learn `ColumnTransformer`.	A shared dictionary containing transformed `X` and unchanged or transformed `y`.
Model	Estimator, `execute_fit`, and optional `cache`.	A deep-copied fitted estimator stored as the node data.
Predict	Validation `test_data`, output method name, and optional positive-class index.	Predictions, probabilities, or one selected probability column.
Metric	A supervised or unsupervised metric wrapper containing the execution method and optional target or feature data.	A structured metric result used for reporting and path selection.

Table 3.3: Concrete payload and output contracts of Accelera Pipe nodes (continued)

Node	Stored Python payload	Runtime output
Merge	A strategy name, the current implementation supports hard or majority voting.	A sample-wise modal class after converting branch outputs to NumPy arrays.

The native graph validates every proposed edge before adding it. A preprocessing or model node can follow an input or preprocessing node, prediction must follow a model, merge receives prediction leaves, and metric follows prediction or merge. A normal node added after several leaves is copied once per valid leaf, whereas a merge node records all valid leaves as its source vector. Any graph mutation clears the compiled flag so that the topological order is regenerated before the next execution.

The Python `branch` method accepts either one node per branch or an ordered list of nodes. It flattens the corresponding payloads, types, and names before calling `Graph.split`. The C++ method reconstructs each chain using branch offsets. Duplicate sharing is local to one original leaf: a fresh `first_branch_nodes` map is created for every leaf, so nodes from unrelated split points are never merged.

The signature deliberately excludes the user-visible node name. It combines the node type with the string representation of the payload, so differently named instances representing the same first operation can share one node. Only the first position of a branch chain is eligible. Thus, branches `[p1,p2,p3]` and `[p4,p2,p3]` keep separate copies of the later `p2` and `p3`, because their incoming data may differ. A one-node branch is naturally covered because its only node is also its first node.

Algorithm 3.1: Branch construction with conservative first-node sharing

```
Input: current graph leaves L and branch chains B
Output: expanded DAG with duplicate first operations shared

for each leaf l in L do
    first_nodes = empty map
    for each branch chain b in B do
        source = l
        for each operation o at position j in b do
            type = node type recorded by the Python builder
            reject o when type cannot legally follow source
            signature = integer(type) + ":" + string(payload(o))

            if j == 0 and signature exists in first_nodes then
                source = first_nodes[signature]
                continue without creating another node
            end if

            n = create typed node from payload(o)
            if j == 0 then first_nodes[signature] = n end if
            n.create_new_data = (j == 0 and type is PREPROCESS)
            n.copy_input = (j == 0 and preprocessing may mutate input)
            connect source to n
            source = n
        end for
    end for
end for
mark the graph as requiring recompilation
```

3.1.4 Preprocessing Execution, Copy Policy, and Optional Cache

The preprocessing implementation distinguishes creation of a new graph data container from copying the arrays inside that container. The `should_create_new_data` flag creates a new $\{X, y\}$ dictionary at the start of an independent preprocessing branch. Otherwise, the node updates and reuses its source dictionary. The separate `should_copy_input` flag protects mutable arrays before a custom operation is called.

The behavior can be summarized as:

$$D_{\text{out}} = \begin{cases} \text{new dictionary copied from } D_{\text{in}}, & \text{if a new branch starts} \\ D_{\text{in}}, & \text{otherwise} \end{cases}$$

where D_{in} is the source data dictionary and D_{out} is the dictionary used by the current preprocessing node.

The copy decision is conservative for unknown callables. Custom functions are assumed capable of in-place mutation and therefore receive copies of X and y. A Scikit-learn transformer normally receives the original input because its standard `transform` contract returns transformed data. If its shallow parameters expose `copy=False`, or parameter inspection fails, copying is enabled. This avoids unconditional branch copies while still protecting the graph from known or uncertain in-place behavior.

This design gives the graph a balance between safety and memory efficiency. If every preprocessing node copied all arrays, branch execution would become safer but memory usage would grow quickly. If no node copied data, one custom function could accidentally modify

shared data used by another branch. Therefore, the backend copies data only when the node is likely to mutate its input or when a new independent branch requires its own data container.

In practice, this means shared preprocessing can reuse the same data object when it is safe, while branch-specific preprocessing receives isolated data when needed. The result is that different branches can be evaluated without changing each other's inputs, and the selected path remains consistent with the expected machine-learning workflow.

Algorithm 3.2: Preprocess node execution and data ownership

```
Input: source node S and preprocessing payload F
Output: transformed data object attached to node N

read (X, y) from the input node or S.data
if N.copy_input then
    X = X.copy()
    y = None if y is None else y.copy()
end if

if F.func provides fit then
    remember the original unfitted function in the training graph
    transformer = deepcopy(F.func)
    if cache is enabled and matching model/data files exist then
        load transformer and transformed X with joblib
    else
        if graph is a training graph then fit transformer on (X, y) end
if
        X = transformer.transform(X)
        store the fitted transformer in F.func
        if cache is enabled then persist transformer and X end if
    end if
else
    X = F.func(X)
end if

if N.create_new_data then
    N.data = {"X": X, "y": y}
else
    update the shared source dictionary and point N.data to it
end if
```

Caching is disabled by default because hashing and storing large arrays can cost more time and memory than recomputation. When enabled for preprocessing, the cache key is a Joblib hash of the transformer and `X`, the fitted transformer and transformed data are stored separately. Model caching similarly hashes the estimator, `X`, and `y`, then stores the fitted model. Both paths use the project cache directory and are intended for repeated runs with equivalent inputs, not as a replacement for executed-graph persistence.

During prediction, a `PredictNode` walks backward from its fitted model, collects all preprocessing payloads, reverses them into training order, and applies each fitted transformer or custom function to validation data. An executed graph instead obtains new data from its input node. The configured output method is then invoked on the fitted estimator. For `predict_proba`, a

nonnegative `positive_class` selects one probability column after bounds validation.

3.1.5 Executed Graph Creation and Pickle State

Path selection occurs after graph execution because metric values are required to choose a branch. Numeric `max` and `min` strategies inspect the result field of each metric node. A custom strategy receives all metric dictionaries with their node names and returns the selected metric-node name. Selection marks the chosen metric and repeatedly follows its source pointer to the input, `all` marks every node.

Algorithm 3.3: Construction of the fitted executed graph

```
Input: completed training graph G and selection strategy s
Output: compact fitted graph G_star

evaluate s and mark selected_in_path on the retained ancestry
create a new input node for G_star
for each selected non-input node n in G do
    clone n and record old-to-new mapping
    retain fitted data only for MODEL nodes
end for
reconnect every cloned source through the old-to-new mapping
preserve selected topological order when available
remove validation test_data from cloned PREDICT nodes
remove y_true and X from cloned METRIC wrappers
mark G_star as executed
clear arrays and temporary runtime data from training graph G
restore original unfitted preprocessors in G
return G_star
```

This separation allows the original pipeline to be trained again while the executed graph owns the fitted models and fitted preprocessing payloads required for inference. Saving either object uses Python `pickle`. PyBind11 links pickle to `Graph.getState` and `Graph.setState`: every node stores its type, name, Python payload, flags, selected-path state, data, and source-node indices. Loading first recreates typed nodes, then performs a second pass that reconnects indices to node pointers. Compilation state is invalidated after loading so the topological order is safely rebuilt.

3.1.6 Branching and Path Selection

The native backend uses a graph abstraction rather than a linear pipeline. The `Graph` class stores all nodes, tracks whether the graph has already been compiled, supports cloning, and exposes methods for adding nodes, splitting branches, executing, clearing, saving, loading, and enabling parallel execution. The graph computes a topological execution order before running, so each node is executed only after its source nodes have produced their outputs.

Branching is handled by the native `split` operation. Each branch is converted into a chain of graph nodes starting from the current leaf. If the current leaf node is L, then a branch B_i can be represented as:

$$B_i = (L, n_{i1}, n_{i2}, \ldots, n_{ik})$$

where $n_{i1}, n_{i2}, \ldots, n_{ik}$ are the operations added to that branch. This means that all branches

start from the same graph point, but each branch can represent a different candidate workflow, such as a different preprocessing strategy, model, prediction step, or metric evaluation path.

After branch execution, the graph selects the required path according to the configured strategy. If $P = \{B_1, B_2, \ldots, B_m\}$ is the set of candidate branches and $M(B_i)$ is the metric value of branch B_i, then a maximization strategy can be represented as:

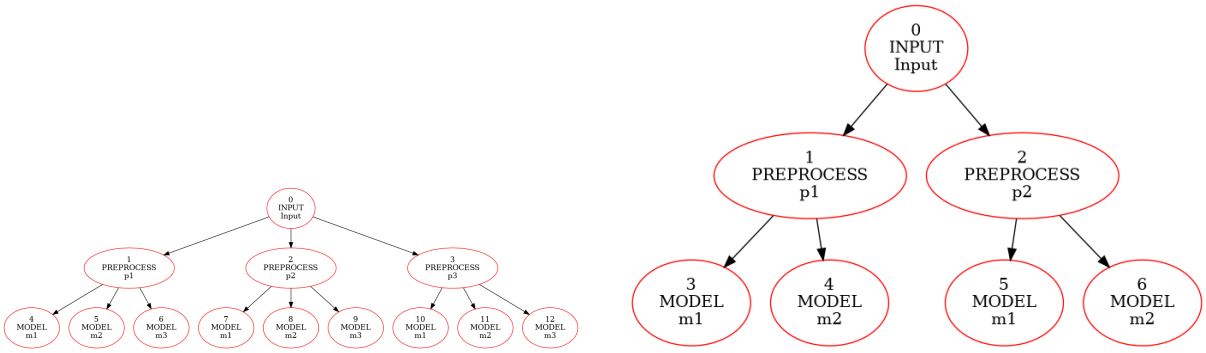
$$B^* = \arg\max_{B_i \in P} M(B_i)$$

For minimization metrics, the same idea is applied using $\arg\min$. The selected branch B^* is then used to construct the compact executed graph used later for inference.

A useful optimization in branch construction is duplicate first-node sharing. If several branches begin with the same first node, the graph creates that first node once and connects later branch consumers to its output. The sharing condition can be written as:

$$n_{11} = n_{21} = \cdots = n_{m1}$$

where n_{i1} is the first node of branch B_i. This is safe because all such branches receive the same original input. The optimization is deliberately conservative: later repeated nodes are not merged automatically because their inputs may already differ due to earlier branch operations.



(a) Before duplicate first-node sharing

(b) After duplicate first-node sharing

Figure 3.2: Branch graph before and after sharing equivalent first branch nodes.

3.1.7 Intermediate Memory Management

Memory management is an important part of the Accelera Pipe design because branch-heavy training can create many temporary arrays. If every intermediate output is kept until the end of execution, the graph may consume unnecessary memory, especially when several preprocessing and prediction branches are evaluated together. To reduce this overhead, the backend tracks how many downstream nodes still need each source output.

For a source node S, the remaining-consumer count is:

$$C(S) = |\{N \mid S \to N \text{ and } N \text{ has not executed yet}\}|$$

This count represents the number of unfinished nodes that still depend on the output of S. After a downstream consumer finishes, the counter is updated as:

$$C(S) \leftarrow C(S) - 1$$

When the counter reaches zero and the source node is safe to clear, the temporary output can be released:

$$C(S) = 0 \Rightarrow \text{release}(S)$$

This policy reduces memory pressure without changing the selected path, metric results, or final predictions. Model and metric nodes are handled more carefully because they may contain fitted estimators or evaluation state, while temporary input and preprocessing outputs can often be cleared after their final consumer has completed. This is especially useful when several branches reuse the same early preprocessing result, because the graph can keep shared data only while it is still needed. After all dependent branches finish, the backend can release the shared output and continue execution with a smaller memory footprint. In this way, memory management becomes part of the graph execution logic rather than a manual responsibility left to the user.

Algorithm 3.4: Consumer-count based intermediate release

```
Input: executed node N, remaining consumer table C
Output: graph with unused temporary data released

1:  for each source node S that feeds N do
2:      C[S] = C[S] - 1
3:      if C[S] == 0 and S is releasable then
4:          clear the temporary data stored by S
5:      end if
6:  end for
```

Algorithm 3.4 summarizes the release mechanism. The backend does not randomly clear intermediate values, it clears a source output only after all of its consumers have finished. This keeps execution correct while allowing the graph to reuse memory during long or branch-heavy training runs.

3.1.8 Parallel Graph Execution

The graph can run independent nodes in parallel, but a node can start only after all of its parents have finished. Therefore, parallelism is controlled by both dependency constraints and hardware-resource constraints. For any node N, the readiness condition is:

$$\text{ready}(N) \iff \forall P \in \text{Parents}(N),\ \text{finished}(P) = \text{true}$$

This equation means that a node is executable only when every parent node has already produced its output. If two ready nodes do not depend on each other, they can be scheduled concurrently.

The backend also limits the number of simultaneously running CPU nodes. If T_{cpu} is the number of available CPU execution slots, then the number of running CPU nodes must satisfy:

$$|\text{Running}_{\text{cpu}}| \leq T_{\text{cpu}}$$

GPU execution is handled more conservatively using a single GPU slot:

$$|\text{Running}_{\text{gpu}}| \leq 1$$

This prevents unsafe concurrent GPU access while still allowing CPU-side graph branches to execute in parallel. The number of CPU threads is based on hardware concurrency but can be overridden using the `ACCELERA_NUM_THREADS` environment variable. During execution, the scheduler repeatedly selects ready nodes whose resource requirements can be satisfied. This allows independent CPU branches to make progress without waiting for unrelated parts of the graph. At the same time, dependency checks preserve the same logical result as a valid sequential topological execution. Therefore, parallel execution improves resource utilization without changing the semantics of the pipeline.

Algorithm 3.5: Parallel graph execution scheduling

```
Input: topological execution order T
Output: completed graph execution or reported error

1:  Initialize finished and started state for all nodes
2:  Build remaining-consumer counts for intermediate outputs
3:  Create limited CPU worker slots and one GPU execution slot
4:  while unfinished nodes still exist do
5:      for each node N in T do
6:          if N has not started and all parent nodes are finished then
7:              Acquire a CPU slot or the GPU slot depending on N
8:              Mark N as started
9:              Launch N asynchronously
10:             After N finishes:
11:                 Mark N as finished
12:                 Release unused source outputs
13:                 Release the CPU or GPU slot
14:         end if
15:     end for
16: end while
17: Wait for all launched tasks
18: Rethrow any stored node execution error
```

Algorithm 3.5 summarizes the scheduling logic used by the native backend. The scheduler scans the topological execution order and starts only nodes whose parents have already finished. Ready nodes are launched asynchronously, but each node must first acquire the appropriate CPU or GPU execution slot. After a node finishes, the backend marks it as completed, releases unused source outputs, and wakes the scheduler so that newly ready child nodes can start.

Finally, the executed graph must not retain training-only data. After training, the selected path is cloned, validation data stored inside prediction nodes is cleared, metric target values are removed, and temporary arrays in the training graph are released. Both unexecuted and executed graphs can be saved as one pickle file. During loading, the native graph rebuilds the nodes and reconnects them using stored source indices. This makes the graph serializable even though part of the implementation is native C++.

3.1.9 Design Constraints

The Accelera Pipe module has several design constraints that guide how the graph is constructed, executed, and reused. The first constraint is that the internal workflow must always remain a valid directed acyclic graph. Since execution depends on topological ordering, cyclic dependencies are not allowed. For any edge from node u to node v, the execution order must satisfy:

$$u \to v \Rightarrow \tau(u) < \tau(v)$$

where τ represents the topological order of the graph. This guarantees that every node is executed only after its required source nodes have already produced their outputs.

The second constraint is node-type validity. Not every node can be connected to any other node. For example, a prediction node must follow a model node, a metric node must follow a prediction or merge node, and a merge node should combine compatible prediction outputs.

These restrictions preserve the semantic meaning of the pipeline and prevent invalid workflows from reaching the execution stage.

The third constraint is safe memory management. Since branch-heavy workflows can produce many temporary arrays, the backend must release intermediate data only when it is no longer needed. For a source node S, the remaining-consumer count is:

$$C(S) = |\{N \mid S \to N \text{ and } N \text{ has not executed yet}\}|$$

The output of S can be cleared only when $C(S) = 0$ and the node is safe to release. This reduces memory pressure without changing the final selected path or prediction results.

The fourth constraint is parallel execution safety. Independent nodes may run in parallel, but a node can start only after all of its parent nodes have completed:

$$\text{ready}(N) \iff \forall P \in \text{Parents}(N),\ \text{finished}(P) = \text{true}$$

This prevents downstream nodes from reading incomplete data. The backend also uses limited CPU worker slots and a separate GPU slot to avoid unsafe resource sharing during execution.

Finally, the executed graph must not retain training-only data. After training, the selected path is cloned into an `ExecutedGraph`, validation inputs are removed, metric targets are cleared, and temporary arrays are released. This keeps the saved inference graph compact and focused only on fitted transformations and models.

3.2 Code Parallelizer Module

3.2.1 Functional Description

The Code Parallelizer module transforms supported loop-heavy source code into optimized C++ code annotated with OpenMP pragmas. It is designed to reduce the execution time of custom preprocessing functions and standalone code snippets by identifying loops that can be safely executed in parallel. The module accepts different input forms: a file path, a source string, a custom Python function, or a custom transformer instance. Regardless of the input form, the goal is to normalize the input into a C++ loop-analysis path, detect parallelization opportunities, and return either parallelized source code or a compiled Python-callable native function.

The transformation can be described as:

$$P' = \mathcal{T}(P)$$

where P is the original user program, P' is the optimized version, and $\mathcal{T}$ represents the complete parallelization process. This process includes input normalization, Python-to-C++ conversion when needed, loop extraction, feature analysis, safety validation, OpenMP pragma insertion, and optional native compilation. The main requirement is that the optimized program must preserve the behavior of the original program:

$$P(X) = P'(X)$$

for the same valid input X. Therefore, the module treats parallelization as a performance optimization, not as a semantic change.

For Python input, the module first lowers a restricted Python subset into C++. This conversion is intentionally limited to predictable language constructs such as arithmetic expressions, assignments, `for range(...)` loops, conditionals, function definitions, array-style indexing, and simple calls. The resulting C++ code is then analyzed by the same loop extraction path used

for native C/C++ input. This avoids building two separate parallelization pipelines and ensures that both Python and C++ inputs are judged using the same loop features and safety checks.

After normalization, the module extracts loop information using native Clang AST bindings. Each loop L_i is represented by its source code and source-line range:

$$L_i = (\text{code}_i,\ \text{start}_i,\ \text{end}_i)$$

The Python orchestration layer then computes a fixed loop-feature representation:

$$\mathbf{x}_i = \phi(L_i)$$

where ϕ is the feature extraction function and $\mathbf{x}_i$ is the feature vector for loop L_i. This vector summarizes important properties such as reductions, array writes, loop nesting, branches, function calls, arithmetic operations, and possible dependencies.

The decision for each loop can be represented as:

$$d_i \in \{\text{sequential},\ \text{parallel\_for},\ \text{reduction}\}$$

where sequential means the loop is left unchanged, parallel_for means a normal OpenMP loop pragma can be inserted, and reduction means the loop can be parallelized with a valid reduction clause.

The decision is not based only on the classifier. The module first applies local rules that can recognize supported reductions, independent array writes, and consecutive nested loop patterns. Only unresolved loops are sent to a classifier service. This makes the system faster for simple safe patterns and more conservative for unclear cases. The final decision can be viewed as:

$$d_i = \text{SafetyRules}(\text{Classifier}(\mathbf{x}_i), L_i)$$

This means that the classifier suggests a decision, but the rule-based stage can reject or adjust it if the loop contains unsafe behavior.

If either the rule-based path or the classifier selects the loop and the final safety checks pass, the module injects an OpenMP directive such as `#pragma omp parallel for`. For reduction-like loops, it can also emit a reduction clause when the reduction variable is valid. The expected benefit can be measured using speedup:

$$S = \frac{T_{\text{original}}}{T_{\text{optimized}}}$$

where T_{original} is the runtime before optimization and $T_{\text{optimized}}$ is the runtime after optimization. A value of $S > 1$ means the optimized version is faster.

The module has fail-safe behavior. If the input cannot be converted, analyzed, classified, compiled, or loaded as a native extension, the system falls back to the original Python function or leaves the code unmodified. This is important because unsafe parallelization can produce incorrect output even if the code compiles successfully. Therefore, the Parallelizer only changes the execution strategy when the transformation is considered safe, otherwise, the original behavior is preserved.

3.2.2 Module Components

The Code Parallelizer is decomposed into cooperating components: input normalization, Python-to-C++ conversion, Clang-based loop extraction, feature extraction and vectorization, classification with rule-based validation, and OpenMP/native-extension generation.

Table 3.4: Main components of the Code Parallelizer module

Component	Responsibility
Parallelization Orchestrator	Receives source strings, file paths, custom functions, and transformer instances. It coordinates conversion, loop extraction, feature analysis, classification, pragma insertion, caching, compilation, and fallback behavior.
Python-to-C++ Converter	Translates a restricted subset of Python into C++ so Python preprocessing logic can enter the same static loop-analysis pipeline as native C/C++ code.
Loop Extraction Engine	Uses a compiler-style front end to locate loops, record source ranges, and expose structured loop information for feature extraction and rewriting.
Feature Extraction and Embedding Generator	Computes structural, dependency, memory-access, control-flow, arithmetic, and reduction features, then maps them into a fixed-size numeric representation.
Classifier and Safety Resolver	Applies deterministic rules first and requests a classifier prediction only for unresolved loops. It decides whether a loop remains sequential, receives a parallel-for pragma, or receives a reduction pragma.
OpenMP Rewriter and Native Compiler	Inserts the selected OpenMP directive, wraps optimized C++ into a Python-callable extension when needed, builds the native module, and returns the optimized callable.
Caching Layer	Stores processed source and compiled native functions using source-based hashes to avoid repeated conversion and compilation.

Figure 3.3 summarizes the overall data flow. In general, supported inputs are normalized into a common C++ loop-analysis path, while callable Python inputs additionally proceed to PyBind11 extension generation.

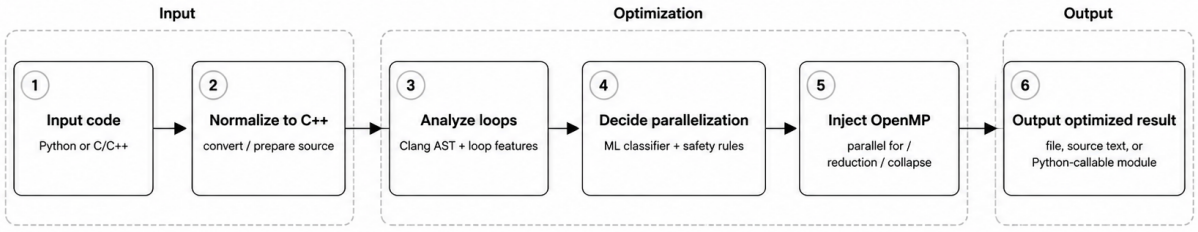


Figure 3.3: Code Parallelizer module workflow

3.2.3 Input Dispatch and Source Normalization

The public `parallelize` method is a dispatcher rather than one large conversion routine. Its behavior depends on the runtime type of the input, as shown in Table 3.5. This separation keeps file output, in-memory source transformation, and callable compilation as distinct operations while reusing the same loop-selection functions.

Table 3.5: Parallelizer input dispatch and returned output

Input form	Implementation path	Output
Existing file path	Read the file, convert it when the suffix is `.py`, extract, select, and rewrite loops.	A `parallelized_<stem>.c` file in the requested output directory or the repository root.
Source string	Try Python AST parsing, successful Python source is converted to C++, while other source continues as C/C++.	Parallelized C++ text returned in memory.
Custom Python function	Capture restorable source, bundle helpers, transform and compile the source, then attach the runtime native callable.	A pickle-compatible `SourceBackedFunction`, original Python execution is retained on failure.
Custom transformer	Optimize only a `transform` method defined directly by the custom class.	The same transformer instance with the method replaced when compilation succeeds.
Other Python object	No supported dispatch rule is applied.	The original object is returned unchanged.

For a file, extracted loops are serialized to a JSON record containing the loop type, start line, end line, and source text. The JSON filename is based on an MD5 hash of the normalized source. In-memory code avoids this intermediate file and converts the native `LoopInfo` objects directly to dictionaries. Both paths then call the same selection and line-rewriting routines.

Custom functions require additional source handling because a compiled runtime callable cannot normally be pickled as portable Python source. `SourceBackedFunction` stores the function name and normalized source, and recreates the Python function with `exec` during unpickling. Referenced top-level helper functions are discovered recursively through the function's global namespace and bundled before the outer function. Nested helper functions are removed from the outer body, ordered by their call dependencies, and emitted as top-level functions before conversion. The implementation rejects closures, captured outer variables, duplicate helper names, and recursive nested helpers, because those cases cannot be represented by the current stateless C++ path. Simple lambdas can be preserved as source for pipeline serialization, but the current Python-to-C++ subset does not compile lambda expressions, so callable optimization falls back to Python for that case.

Algorithm 3.6: Normalization of a Python callable for compilation

```
Input: custom Python function f
Output: restorable source S and compilation source C

reject f when it captures closure variables
obtain and dedent inspect.getsource(f)
parse the source into a Python AST
recursively collect referenced top-level helper functions
bundle helpers before f and save the bundle as S

find nested function definitions inside f
for each nested helper h do
    reject captured outer variables
    record calls from h to other nested helpers
end for
reject duplicate names or recursive helper dependencies
topologically order and hoist nested helpers
remove their original nested definitions from f
emit top-level helpers followed by f as C
```

3.2.4 Deterministic Pragma Generation Without AI

The classifier service is optional for the currently recognized rule patterns. For every extracted `for` loop, `_select_parallel_loops` first calls the resolver with the class `none`. The resolver can upgrade this class locally without any network request or learned model. Therefore, the system can generate pragmas when the classifier is offline, times out, returns an error, or is deliberately bypassed, provided that a loop matches one of the implemented deterministic patterns.

The rule-only path currently recognizes three families:

1. **Scalar reduction update.** The extractor recognizes scalar `v += expression`, `v = v + expression`, `v *= expression`, and `v = v * expression`. A candidate is rejected as a scalar reduction when the same name is indexed as an array. The pragma emitter supports OpenMP `reduction(+ : v)` and `reduction(* : v)` when `v` is not declared inside the loop. Maximum and minimum calls are recorded as classifier features, but the current emitter does not yet construct OpenMP `max` or `min` clauses.

2. **Independent array write.** A loop is considered an independent write candidate when at least one indexed assignment is found and the local heuristics find no previous-iteration access of the form `A[i-1]`, no indirect indexing such as `A[B[i]]`, and no early `break`, `continue`, or `return`. This rule produces a plain `parallel for` directive.

3. **Consecutive nested loops.** The parser counts directly consecutive `for` headers, allowing whitespace and braces between them. A depth greater than one selects the outer loop and produces `collapse(depth)`. When both outer and inner ranges appear in the selected-loop list, the rewriting stage skips the inner range so that it does not create nested duplicate pragmas.

Algorithm 3.7: Classifier-free rule-based OpenMP generation

```
Input: normalized C++ source P and extracted for-loops L
Output: source P_rule containing deterministic OpenMP pragmas

selected = empty list
for each loop l in L do
    features = extract_features(l.code)

    if l contains a supported scalar reduction update then
        class = reduction
    else if l writes an array and has no detected carried dependency,
            indirect access, or early exit then
        class = parallel_for
    else if consecutive_for_depth(l.code) > 1 then
        class = parallel_for
    else
        class = none
    end if

    if class is not none then append (l, class) to selected end if
end for

for each selected loop ordered by its source range do
    skip it when its range is contained by a selected outer loop
    clauses = empty list
    if consecutive depth > 1 then add collapse(depth) end if
    if external scalar += reduction then add reduction(+ : variable) end
if
    if external scalar *= reduction then add reduction(* : variable) end
if
    replace the original source range with pragma, clauses, and loop text
    update the line-offset shift caused by inserted pragma lines
end for
return rewritten source
```

No classifier endpoint is contacted for a loop selected by these rules. If the rules return `none`, the implementation attempts to send the 40-dimensional feature vector to the remote classifier. A classifier failure is caught and converted back to `none`, the resolver is then called again, so unresolved loops remain unchanged. If the classifier returns a non- `none` class, the current resolver accepts that class and the pragma emitter constructs the directive. Consequently, the deterministic path is a useful offline fallback and pre-classifier shortcut, but the current implementation should not be described as a complete dependence proof for all classifier-selected loops. Extending the rejection rules for aliasing, side-effecting calls, and general loop-carried dependencies remains future work.

3.2.5 Feature Vector and Source Rewriting Details

Before classification, comments are removed and structural features are computed from the loop text. The 40-dimensional vector contains Boolean flags for reductions, carried dependencies, control flow, constant bounds, nesting, RAW/WAR indicators, indirect access, early exit, computable trip count, and vectorizability. Other positions encode bounded counts for array reads and writes, nesting depth, branches, calls, arithmetic operations, memory operations, read/write variables, and reduction variables. Array names are mapped into fixed five-dimensional hashed summaries so the request size remains constant across loops.

For a loop L_i, the feature extraction stage can be represented as:

$$\mathbf{x}_i = \phi(L_i), \qquad \mathbf{x}_i \in \mathbb{R}^{40}$$

where ϕ is the feature extraction function and $\mathbf{x}_i$ is the fixed-size feature vector sent to the decision stage. This fixed representation is useful because loops may have different lengths and different numbers of variables, but the classifier always expects the same input size.

The classifier is called with an HTTP POST request whose JSON body contains the vector under `embedding`. The configured timeout bounds network delay. The expected response provides one of `none`, `parallel_for`, or `reduction`. Rule-selected loops bypass this request entirely, which also makes common reductions and indexed writes available when no Internet connection is present. The final loop decision can be written as:

$$d_i = \begin{cases} \text{rule}(L_i), & \text{if a safe rule matches} \\ \text{classifier}(\mathbf{x}_i), & \text{otherwise} \end{cases}$$

where d_i is the selected action for loop L_i. This keeps simple safe cases fast while still allowing the classifier to help with less obvious loops.

Rewriting uses the source ranges reported by Clang rather than searching for the loop text globally. For each selected loop, the one-based start and end lines are converted to Python slice indices. The generated pragma-plus-loop segment replaces that slice. A running `shift` records how many lines earlier insertions added, ensuring that the ranges of later loops still address the correct code. This can be summarized as:

$$s'_i = s_i + \Delta, \qquad e'_i = e_i + \Delta$$

where s_i and e_i are the original start and end lines, and Δ is the current line shift caused by previously inserted pragmas. A selected loop is skipped when its complete range lies inside another selected range, preventing duplicate inner directives.

3.2.6 Native Wrapper, Compilation, and Cache Layers

Callable optimization adds a second transformation after pragma insertion. `to_numpy_code_cpp` removes the generated `main`, strips template declarations, maps temporary template types to `double`, and rewrites the selected one-argument function to accept a `py::array_t<double>`. The wrapper creates a mutable unchecked two-dimensional view, translates `len(X)` and `len(X[i])` to array shape queries, and converts two-dimensional indexing to view calls. This means the current direct native-callable path is intentionally restricted to one named input parameter representing a two-dimensional double array.

This restriction makes the generated native wrapper easier to control and test. The callable input can be viewed as:

$$X \in \mathbb{R}^{m \times n}$$

where m is the number of rows and n is the number of columns. Python indexing such as `X[i][j]` is rewritten to access the corresponding native array view. This allows the optimized C++ function to operate directly on NumPy data while still being called from Python.

The cache layer reduces repeated compilation cost. A source-based hash is used to identify previously processed code:

$$h = H(\text{source})$$

where H is the hashing function. If the same source is optimized again, the module can reuse the cached generated code or compiled native extension instead of repeating conversion and compilation. If compilation or loading fails, the system returns the original Python callable, so optimization failure does not break the pipeline.

Algorithm 3.8: Cached compilation of an optimized Python callable

```
Input: Python function f and restorable source S
Output: compiled callable f_native or original f

C = build compilation source from S and helper dependencies
method_key = hash(cache_version, name, optimization, extension, C)
if method metadata and shared module exist then
    dynamically load and return the cached exported function
end if

C_cpp = convert supported Python AST to C++
P = apply rule/classifier loop selection and insert OpenMP pragmas
module_key = hash(cache_version, name, optimization, extension, P)
wrap P as a PyBind11 NumPy extension named from module_key

if the shared module does not exist then
    compile with C++17, shared-library, and release-definition flags
    add -fopenmp only when P contains an OpenMP pragma
    add Python and PyBind11 include paths
    add PIC and pipe flags on non-Windows systems
    use sccache or ccache when enabled and available
end if

dynamically import the extension and register it in sys.modules
write method metadata that points to the compiled module
return the exported function
on conversion, compilation, or loading failure, return original f
```

There are two persistent cache levels in addition to Python's in-process module table. The processed-code cache hashes the cache-format version and input source and stores rewritten C++. The Python-method cache additionally includes the function name, compiler optimization flag, and platform extension suffix, then stores the generated module name in JSON. The compiled-module name hashes the final C++ text with the same compatibility inputs. Including `COMPILE_CACHE_VERSION` lets developers invalidate old artifacts after changing conversion or wrapper semantics.

The compiler defaults to `c++` on Unix-like systems and searches for Clang on Windows, the `CXX` environment variable can override it. `ACCELERA_CPP_OPT_LEVEL` selects the optimization flag, and `ACCELERA_USE_COMPILER_CACHE=0` disables `sccache` or `ccache`. OpenMP is linked

only when the transformed source actually contains `#pragma omp`, avoiding an unnecessary OpenMP requirement for unchanged serial callables.

3.2.7 Loop Analysis, Classification, and Conversion

The feature extraction stage analyzes both code structure and loop behavior, including reductions, array accesses, dependencies, indirect memory access, early exits, nesting depth, branches, function calls, arithmetic operations, and vectorization potential. These indicators are encoded into a 40-dimensional vector for the classifier. Before any network request, deterministic rules can select recognized reductions, independent array writes, and consecutive nested loops. The classifier is used only when these rules leave the loop unresolved.

Algorithm 3.9: Loop classification and pragma insertion

```
Input: source program P
Output: source program P' with selected OpenMP pragmas

if P is Python source then
    convert the supported Python subset into C++
end if
Extract for-loops using the Clang AST frontend
for each extracted loop L do
    compute structural, memory, dependency, and reduction features
    resolve L using the deterministic rules with initial class none
    if L is still none then
        request the classifier; on failure retain class none
    end if
    if the resolved class is parallel_for then
        insert #pragma omp parallel for and derived clauses
    else if the resolved class is reduction then
        insert #pragma omp parallel for and supported reduction clause
    else
        leave the loop unchanged
    end if
end for
Cache and return the transformed source code
```

The Python-to-C++ converter uses Python's `ast` module and intentionally supports only a small subset suitable for numeric loops. Unsupported syntax raises `ValueError`. Table 3.6 lists the supported operations.

Table 3.6: Supported operations in the Python-to-C++ converter

Category	Supported forms and notes
Constants and names	`None`, bool, int, float, string, variables.
Arithmetic	`+`, `-`, `*`, `/`, `%`, `**`; power uses `std::pow`.
Unary/boolean	unary `+`, unary `-`, `not`, `and`, `or`.
Comparisons	`==`, `!=`, `<`, `<=`, `>`, `>=`; chained comparisons rejected.
Calls/printing	Ordinary calls and `print`; keyword arguments are generally unsupported.
Access	Attribute access and indexing; slices rejected.
Assignment	Single-target assignment and indexed assignment; multi-target rejected.
AugAssign	Simple-name `+=`, `-=`, `*=`, `/=`, and `%=`.
Control flow	`if`/`else` and `return`.
Loops	`for i in range(...)` with one, two, or three range arguments.
Functions	`def f(...)` as C++ template functions; decorators rejected.

Table 3.6 is intentionally narrow. The converter is not a general Python compiler, it is a practical bridge for loop-heavy preprocessing code. By rejecting unsupported constructs early, the Parallelizer avoids silently generating C++ that changes Python semantics. The supported subset is enough for examples such as row normalization, scalar reductions, simple indexing, and range-based nested loops.

3.2.8 Integration with Accelera Pipe

The Code Parallelizer integrates with Accelera Pipe through custom preprocessing nodes. If `Pipeline.preprocess()` receives a custom Python function, the pipeline attempts to optimize it with the Parallelizer before storing it inside the graph. If it receives a custom transformer object, the transformer instance can be optimized through its supported preprocessing method. In both cases, the optimization is best-effort: failed conversion, unsupported syntax, compilation failure, or module loading failure falls back to the original Python callable.

Figure 3.4: Simplified flow for parallelizing a Python method or transformer and using it inside an Accelera Pipe graph node.

Figure 3.4 summarizes the integration at a higher level. The preprocessing callable is converted to C++ when it belongs to the supported subset, the loop analysis stage adds OpenMP parallelism, and the compiled module is attached back to the graph node. The graph therefore keeps the same node structure while replacing eligible Python loops with native execution.

Overall, the Code Parallelizer acts as an optimization layer around user-defined numeric loops. It does not attempt to parallelize all Python programs. Instead, it targets the subset where static analysis, feature extraction, classifier prediction, rule validation, and native compilation can cooperate safely.

3.2.9 Design Constraints

The most important design constraint of the Code Parallelizer is correctness. OpenMP pragmas can improve performance, but an incorrect pragma can produce wrong numerical results. The current deterministic resolver checks a limited set of dependency indicators for its rule-selected loops, including previous-index access, indirect indexing, early exits, and reduction-variable scope. Inner loops are also skipped when an outer selected loop already covers their range, preventing nested duplicate pragma insertion. A non-`none` classifier class is currently accepted by the resolver, so the implementation does not yet provide a complete static safety proof for every classifier-selected loop. This limitation is why output-equivalence testing and conservative Python fallback remain essential.

The second constraint is the restricted Python subset. Full Python semantics are too dynamic to translate safely into C++ using a lightweight source converter. The converter therefore rejects unsupported syntax such as decorators, slicing, chained comparisons, unsupported operators, non-range loop iterators, multi-target assignment, complex dynamic dispatch, and unsupported expression forms. This restriction is a correctness boundary: the system optimizes code only when it can produce predictable C++.

The third constraint is compiler and platform compatibility. Native callable optimization depends on a working C++ compiler, Python include paths, PyBind11 headers, NumPy-compatible array wrapping, and platform-specific shared extension suffixes. The compiler path uses C++17 and adds `-fopenmp` only when the parallelized code actually contains OpenMP pragmas. If conversion, compilation, or module loading fails, the original Python callable is returned.

An earlier generator trial attempted to learn OpenMP pragma generation as a sequence-to-sequence task. This direction was not selected as the primary implementation because the available corpus size was too small for robust code generation and because synthetic examples could introduce distribution bias. A rule-based and feature-driven design was therefore preferred because it is more predictable, auditable, and safer for semantic-preserving OpenMP insertion.

Chapter 4

Experiments and Results

4.1 Controlled Pipeline Comparison

The main `accelera_pipe` experiment compares three implementations of the same four-candidate model-selection task: a manual sklearn loop, sklearn `Pipeline`, and `accelera_pipe`. Each implementation evaluates the same preprocessing alternatives, `StandardScaler` and `MinMax Scaler`. The model alternatives are `LogisticRegression` and `SVC`. Candidate selection is performed on validation accuracy, and final accuracy is reported on the test split only once.

Tables 4.1 and 4.2 are the main scalability results. The benchmark starts with 1,000 total samples and grows to 250,000 total samples. Every row keeps the same protocol: 60% training, 20% validation, and 20% test data. Green highlights the fastest runtime for each sample size, while blue highlights the lowest memory usage.

Table 4.1: Latency scaling in seconds.

Samples	sklearn	sk Pipe	Accelera
1,000	0.06	0.05	0.34
5,000	2.47	2.12	2.02
10,000	2.48	2.48	1.79
50,000	23.96	23.25	14.81
100,000	77.79	77.10	67.04
250,000	803.56	803.01	494.64

Table 4.1 shows the overhead boundary. At 1,000 samples, sklearn Pipeline finishes in 0.05 seconds, while `accelera_pipe` takes 0.34 seconds. This is expected because the native graph has setup, node dispatch, and branch-management overhead that cannot be amortized by such a small workload. Starting from 5,000 samples, the pattern changes: `accelera_pipe` becomes slightly faster than both sklearn baselines, and the advantage increases as the dataset grows. At 50,000 samples, it reduces runtime from 23.25 seconds for sklearn Pipeline to 14.81 seconds. At 250,000 samples, the reduction is much larger: from 803.01 seconds to 494.64 seconds.

Table 4.2 tells the opposite side of the design. sklearn Pipeline has the lowest RSS delta in every row because it runs one candidate pipeline at a time and releases many temporary objects naturally through Python object lifetime. `accelera_pipe` keeps graph nodes, branch outputs, and fitted branch state alive during execution, so its RSS delta is higher. This is why the memory optimizations in the methodology are important: consumer-count release, guarded copy removal, duplicate first-node sharing, and training-graph cleanup directly target the gap

Table 4.2: RSS memory delta scaling in MB.

Samples	sklearn	sk Pipe	Accelera
1,000	1.62	0.16	19.32
5,000	3.62	1.09	21.62
10,000	11.47	2.19	7.25
50,000	126.95	9.75	209.25
100,000	139.70	47.65	300.93
250,000	303.95	28.52	594.86

shown in Table 4.2.

Table 4.3: Largest controlled comparison run.

System	Val.	Test	Time	RSS
sklearn	0.9774	0.9768	803.56	303.95
sklearn Pipe	0.9774	0.9768	803.01	28.52
`accelera_pipe`	0.9774	0.9768	494.64	594.86

Table 4.3 isolates the largest run from the scaling experiment because it is the clearest evidence of the latency benefit. The dataset contained 150,000 training samples, 50,000 validation samples, and 50,000 test samples, each with 25 features. All three systems selected an equivalent `StandardScaler` followed by `SVC` path and achieved the same test accuracy, 0.9768. This means the runtime improvement is not caused by evaluating a different model, skipping validation, or changing the final test protocol. Under identical candidate selection behavior, `accelera_pipe` reduced runtime by 308.91 seconds relative to manual sklearn and 308.36 seconds relative to sklearn Pipeline. The table also makes the current limitation explicit: the faster graph execution pays a memory cost, reaching 594.86 MB RSS delta on this run.

4.2 OpenMP Classifier Evaluation

The Parallelizer classifier is evaluated as a three-class loop classifier. Its output is combined with local rules during pragma generation, but the classifier metrics still describe the quality of the learned loop-feature representation.

The classifier is not expected to be perfect because the final system still applies deterministic checks after prediction. However, Table 4.4 is useful because it shows whether the learned feature vector can separate the three main loop categories. The `parallel_for` class has the highest precision at 0.86, which is important because false positive parallelization can be unsafe. The `reduction` class has lower support, only 775 examples, but still reaches 0.82 recall. This suggests that the feature extractor captures common scalar-reduction patterns well enough for the classifier to help, while the rule layer remains necessary for safety.

The macro and weighted averages are both close to 0.83 F1-score. The macro average treats the three classes equally, so it is sensitive to the smaller `reduction` class. The weighted average follows the dataset distribution, where `none` and `parallel_for` are more common. The closeness of these two averages indicates that the model is not only performing on the majority classes. In the full Parallelizer, these predictions are used as a first signal, then local rules refine the decision before any OpenMP pragma is emitted.

Table 4.4: OpenMP classifier test metrics.

Class	Prec.	Rec.	F1	Support
`none`	0.82	0.85	0.84	3048
`parallel_for`	0.86	0.81	0.83	2696
`reduction`	0.79	0.82	0.80	775
Accuracy			0.83	6519
Macro avg.	0.82	0.83	0.83	6519
Weighted avg.	0.83	0.83	0.83	6519

4.3 Parallelizer Hard-File Evaluation

The hard-file evaluation script parallelizes examples from `data/hard_test_files`, compiles both baseline and generated versions, runs each executable three times, compares outputs, and records timing. Numeric outputs use tolerant comparison because OpenMP reductions can change floating-point accumulation order. Table 4.5 shows the measured results.

Table 4.5: Hard-file Parallelizer evaluation.

File	Base (ms)	Par. (ms)	Speedup
`hard_eval_000.cpp`	1786.28	195.47	9.138×
`hard_eval_001.cpp`	63.98	31.47	2.033×
`hard_eval_002.py`	42.66	7.67	5.563×

Table 4.5 evaluates the generated code as executable programs, not only as text. This matters because a pragma can look correct syntactically while still changing output values or failing to compile. The first file obtains the largest speedup, 9.138×, because its workload benefits strongly from parallel loop execution. The second file still improves by 2.033×, suggesting higher overhead or less exploitable parallelism. The third file begins as supported Python, passes through the Python-to-C++ converter, and still reaches 5.563×, validating both conversion and pragma insertion paths.

The three files were parallelized, compiled, executed, and verified successfully. The report contained 11 extracted loops, 8 gold pragmas, and 8 generated pragmas. Exact pragma matching reached 8 out of 8, with average pragma similarity of 1.0, meaning the generated annotations matched the expected ones. The average measured parallelization latency was 231.53 ms, which is small compared with the runtime savings for long-running kernels. Across the three verified files, the average speedup was 5.578×.

4.4 Pipeline Integration Benchmark

The integration experiment uses a custom row-normalization preprocessing function over an input shape of `(500000, 25)`. The Python implementation contains nested loops that compute the row norm and divide each feature by the norm. The optimized path converts the function to C++, inserts OpenMP, compiles a pybind module, and uses the compiled callable inside `accelera_pipe`.

Table 4.6: Automatic custom preprocessing acceleration inside `accelera_pipe`.

Mode	Samples	Time (s)
Python custom preprocessing function	500,000	6.83
End-to-end optimized run, first measured process	500,000	5.88
End-to-end optimized run, warmed method and module cache	500,000	0.08

The integration benchmark in Table 4.6 reports the direct `examples/parallel_accpipe.py` run after normalizing example execution to the repository root. The pure Python preprocessing function takes 6.83 seconds on 500,000 samples. The first optimized process in the measured sequence takes 5.88 seconds because process-local setup and cache lookup still appear in the end-to-end path. The immediate repeated run takes 0.08 seconds after the Python-method cache and compiled-module cache are both warm. The example validation confirmed that the optimized output matched the Python output. This result shows that the integration benefits from two cache layers: one that avoids repeating Python-to-C++ conversion and OpenMP insertion, and one that avoids recompiling the pybind extension.

4.4.1 Direct Example Verification Across Operating Systems

The final verification step runs the same demonstration scripts used by the project Makefile. These examples are important because they exercise the modules as a user would run them: graph pipeline execution, automatic preprocessing optimization, standalone source parallelization, and save/load persistence. The Linux and Windows measurements should not be compared as hardware-normalized benchmarks, they are environment verification runs. Their purpose is to confirm that the examples complete, preserve outputs or accuracy, and expose where compilation and operating-system overhead appear. This also helps verify that the framework behaves consistently outside isolated unit tests, where file paths, compiler setup, cache behavior, and process startup time can affect execution. Therefore, the results are used mainly as practical evidence that the full workflow remains usable across operating systems, rather than as a strict performance comparison between Linux and Windows.

Table 4.7: Linux direct example-script verification runs.

Example	Baseline time (s)	Optimized time (s)	Validation
`accpipe_demo.py`	2.48	1.79	same test accuracy, 0.9260
`parallel_accpipe.py`, run 1	6.83	5.88	outputs match
`parallel_accpipe.py`, run 2	6.83	0.08	outputs match
`code_parallelizer_demo.py`, run 1	5.81	0.014	outputs match
`code_parallelizer_demo.py`, run 2	6.05	0.015	outputs match
`save_load.py`, run 1	1.89	2.69 / 2.65	same accuracy, 0.268
`save_load.py`, run 2	2.53	2.20 / 2.14	same accuracy, 0.268

Table 4.7 summarizes the Linux examples invoked by `examples/Makefile`, but each file was run directly rather than through Make so that the timing for each demonstration could be read separately. Cache-sensitive files were repeated. The standalone `code_parallelizer_demo.py` result measures Python script execution against the generated OpenMP executable for `hard_eval_002.py`. The C path also reported about 0.26–0.27 seconds of compilation time outside the listed executable runtime. The save/load example verifies persistence rather than pure acceleration: both sklearn and Accelera load persisted state and produce the same accuracy, while the second Accelera run is slightly faster than the sklearn baseline.

Table 4.8: Windows direct example correctness and path timing.

Example	Run/cache state	Baseline (s)	Accelera/C path (s)	Validation
`accpipe_demo.py`	normal	1.75 / 1.78	1.03	same test accuracy, 0.9260
`parallel_accpipe.py`	isolated cache, run 1	6.66	6.00	outputs match
`parallel_accpipe.py`	isolated cache, run 2	6.58	0.09	outputs match
`code_parallelizer_demo.py`	isolated cache, run 1	5.59	1.04	outputs match
`code_parallelizer_demo.py`	isolated cache, run 2	5.61	1.05	outputs match
`save_load.py`	normal	0.57	0.51 / 0.53	same accuracy, 0.268

Table 4.9: Windows compilation and process overhead.

Example	Run/cache state	Compile/link (s)	Wall time (s)	Interpretation
`accpipe_demo.py`	normal	–	6.53	Graph example startup
`parallel_accpipe.py`	isolated cache, run 1	included	14.37	Cold compile path
`parallel_accpipe.py`	isolated cache, run 2	cached	8.33	Warm cache path
`code_parallelizer_demo.py`	isolated cache, run 1	0.87	8.74	Temporary executable
`code_parallelizer_demo.py`	isolated cache, run 2	1.04	9.00	Temporary executable
`save_load.py`	normal	–	2.71	Persistence example

Tables 4.8 and 4.9 report the same Makefile example set on Windows. The first table keeps the user-visible comparison: baseline runtime, optimized path runtime, and validation result. The second table separates compilation and process-level overhead so the timing table can remain readable. The examples were run one by one in Makefile order. The cache-sensitive examples were then repeated under an isolated Accelera cache so that the first run represents cold-cache behavior and the second run represents warm-cache behavior. The Windows build path also exercises the platform changes made for the Parallelizer: CMake searches for LLVM/Clang, attempts automatic installation through Windows package managers when needed, and the runtime compiler uses the LLVM `clang++` path without Linux-only flags.

The strongest cache effect appears in `parallel_accpipe.py`. The cold end-to-end Accelera path takes 6.00 seconds because it must extract the Python function source, convert it to C++, select safe loops, generate OpenMP pragmas, compile a pybind extension, link it, import it, and attach the compiled callable to the preprocessing node. The immediate warm-cache run takes 0.09 seconds because the processed-code cache, method cache, and compiled-module cache avoid repeating those setup stages. This verifies that Windows execution benefits from the same cache design as Linux, even though the process-level wall time remains higher.

The Windows `code_parallelizer_demo.py` runs mainly measure end-to-end toolchain latency rather than pure loop speed. Each run rebuilds a temporary executable, so the reported wall time includes compilation, process startup, OpenMP initialization, dynamic loading, and executable creation. The generated program itself reports only about 0.024–0.025 seconds for the timed main-body work, while Linux has lower launch and linking overhead, making its runtime closer to the measured kernel work.

Chapter 5

User Guide

This chapter presents the shortest practical path for using the two modules. The repository must first be installed in a Python environment and the native bindings must be built with CMake. The Parallelizer additionally requires a supported C++ compiler, LLVM/Clang, and OpenMP. The examples assume that the training, validation, and test arrays already exist.

5.1 Using Accelera Pipe

An Accelera Pipe workflow is created by chaining preprocessing, model, prediction, and metric nodes. Objects passed with `branch=True` become branch descriptions, and `branch()` inserts their alternatives into the graph. The following example compares two scalers under one classifier and selects the path with the highest validation accuracy.

Code Example 5.1: Training and selecting an Accelera Pipe graph

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from accelera.src.accelera_pipe.core.pipeline import Pipeline

pipe = Pipeline()
pipe.branch(
    "preprocessing",
    pipe.preprocess("standard", StandardScaler(), branch=True),
    pipe.preprocess("minmax", MinMaxScaler(), branch=True),
).model(
    "model", LogisticRegression(max_iter=1000)
).predict(
    "validation_prediction", test_data=X_val
).metric(
    "validation_accuracy", "accuracy_score", y_true=y_val
)

validation_results, executed_graph = pipe(
    X_train, y_train, select_strategy="max"
)
test_results = executed_graph(X_test, y_true=y_test)
```

The training call returns the graph outputs and an `ExecutedGraph` containing the fitted selected path. The selection strategy may be `all`, `max`, `min`, or a function supplied through

`custom_strategy`. Parallel graph execution is enabled by default, it can be disabled with `disable_parallel_execution()` for debugging. Node caching is disabled by default and should be enabled only when repeated executions reuse the same expensive inputs.

Both the training recipe and the fitted inference graph can be stored in one pickle file. A saved `Pipeline` can be trained later, whereas a saved `ExecutedGraph` can perform inference immediately.

Code Example 5.2: Saving and loading graph objects

```
from accelera.src.accelera_pipe.core.executed_graph import ExecutedGraph
from accelera.src.accelera_pipe.core.pipeline import Pipeline

pipe.save("training_graph.pkl")
loaded_pipe = Pipeline.load("training_graph.pkl")

executed_graph.save("inference_graph.pkl")
loaded_graph = ExecutedGraph.load("inference_graph.pkl")
predictions = loaded_graph(X_test, y_true=y_test)
```

5.2 Using the Code Parallelizer

The `parallelize()` method dispatches according to its input. A source file path produces `parallelized_<name>.c` in the repository root or in the supplied `output_dir`. An in-memory Python or C/C++ source string returns transformed C++ text. A supported Python function returns a callable backed by the compiled native module.

Code Example 5.3: Parallelizing files and source strings

```
from accelera.src.parallelizer.parallelizer import parallelizer

parallelizer.parallelize(
    "data/hard_test_files/hard_eval_002.py",
    output_dir="generated",
)

code = """
int total = 0;
for (int i = 0; i < 1000; ++i) total += i;
"""
parallelized_code = parallelizer.parallelize(code)
```

Custom preprocessing functions can be optimized directly. The returned object retains normal Python-callable behavior. If source extraction, conversion, analysis, compilation, or import fails, the original Python implementation is kept and a warning is printed.

Code Example 5.4: Optimizing a Python function and using automatic integration

```
def square_values(X):
    for i in range(len(X)):
        for j in range(len(X[i])):
            X[i][j] = X[i][j] * X[i][j]
    return X

optimized = parallelizer.parallelize(square_values)
result = optimized(X_test.copy())

# Pipeline.preprocess performs the same optimization attempt automatically.
pipe = Pipeline()
pipe.preprocess("square_values", square_values)
```

Generated and compiled outputs are cached, so the first call may include conversion and compilation cost while later calls can reuse the native module. Users should verify output equality before interpreting speedup and should measure cold-cache, warm-cache, and kernel time separately.

Chapter 6

Gained Experience and Future Work

6.1 Gained Experience

The work provided practical experience in graph execution, native compilation, and performance evaluation. Accelera Pipe showed that safe branch parallelism requires explicit data ownership, isolation of mutable inputs, and delayed release of temporary arrays until their final consumer finishes. It also clarified the different state required by training and inference graphs.

The Parallelizer covered AST conversion, Clang loop extraction, dependence analysis, OpenMP generation, PyBind11 compilation, caching, and fallback. It showed both the value of deterministic offline rules and the need for stronger validation around classifier-selected loops. The experiments also reinforced fair comparison: equivalent inputs and metrics, output validation, and separate reporting of compilation, warm-cache execution, memory, and operating-system overhead.

6.2 Future Work

6.2.1 Accelera Pipe

Future work should add copy-on-write data, resource-aware scheduling, common prefix sharing, per-node telemetry, and versioned graph persistence.

6.2.2 Code Parallelizer

The converter should support more Python and NumPy patterns, while stronger dependence and cost analysis can expand OpenMP safely and skip unprofitable loops.

6.2.3 Integrated Direction

The long-term objective is a graph-aware choice between Python, serial C++, and OpenMP that preserves fallback, reproducibility, and output equivalence.

Chapter 7

Conclusion

The two examined modules address complementary bottlenecks. `accelera_pipe` improves workflow-level execution by representing candidate pipelines as a graph, running branches in parallel, selecting paths through metrics, reusing executed graphs for inference, and reducing temporary data retention. The Parallelizer improves loop-level execution by converting supported Python code to C++, extracting loops, classifying OpenMP opportunities, and compiling supported custom preprocessing functions into native Python-callable modules.

The current results show that the graph executor can match sklearn accuracy while reducing large benchmark latency, and that the Parallelizer can produce substantial speedups for both standalone hard loop files and integrated preprocessing kernels. Future work should expand the literature review, broaden the Python-to-C++ subset, improve memory usage during branch-heavy searches, and refine compile-cache policy for interactive workflows.

Bibliography

[1] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[2] T. Kadosh, N. Hasabnis, P. Soundararajan, V. A. Vo, M. Capota, N. Ahmed, Y. Pinter, and G. Oren, "OMPar: Automatic Parallelization with AI-Driven Source-to-Source Compilation," arXiv preprint arXiv:2409.14771, 2024.